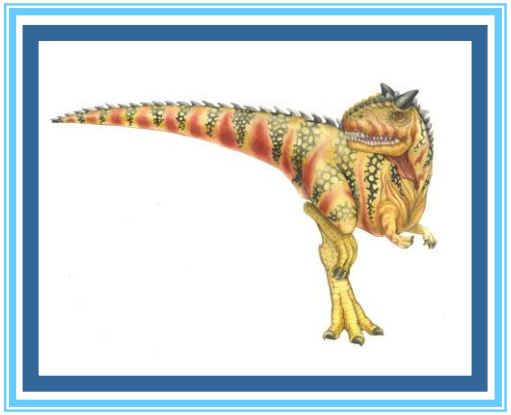
Capítulo 3: Procesos



Silberschatz, Galvin and Gagne ©2018



Agenda

- Concepto de proceso
- Programación de Procesos
- Operaciones en procesos
- Comunicación entre procesos
- IPC en sistemas de memoria compartida
- IPC en sistemas de paso de mensajes
- Ejemplos de sistemas IPC
- Comunicación en sistemas cliente-servidor





Objetivos

- Identificar los componentes separados de un proceso e ilustrar cómo están representados y programados en un sistema operativo.
- Describir cómo se crean y finalizan los procesos en un sistema operativo, incluido el desarrollo de programas utilizando las llamadas apropiadas al sistema que realizan estas operaciones.
- Describir y contrastar la comunicación entre procesos utilizando la memoria compartida y el paso de mensajes.
- **Diseñar programas que usen tuberías y memoria compartida POSIX para realizar la comunicación entre procesos.**
- **Describir la comunicación cliente-servidor utilizando sockets y llamadas a procedimientos remotos.**
- **Diseñar módulos de kernel que interactúen con el sistema operativo Linux.**





Concepto de proceso

- Un sistema operativo ejecuta una variedad de programas que se ejecutan como un proceso.

- Proceso: un programa en ejecución; la ejecución del proceso debe progresar de manera secuencial. Sin ejecución paralela de instrucciones en un solo proceso
- Partes múltiples
  - El código del programa, también llamado sección de texto.
  - Actividad actual incluyendo contador de programa, registros de procesador
  - Pila que contiene datos temporales
  - Parámetros de función, direcciones de retorno, variables locales.
  - Sección de datos que contiene variables globales
  - Contiene memoria asignada dinámicamente durante el tiempo de ejecución





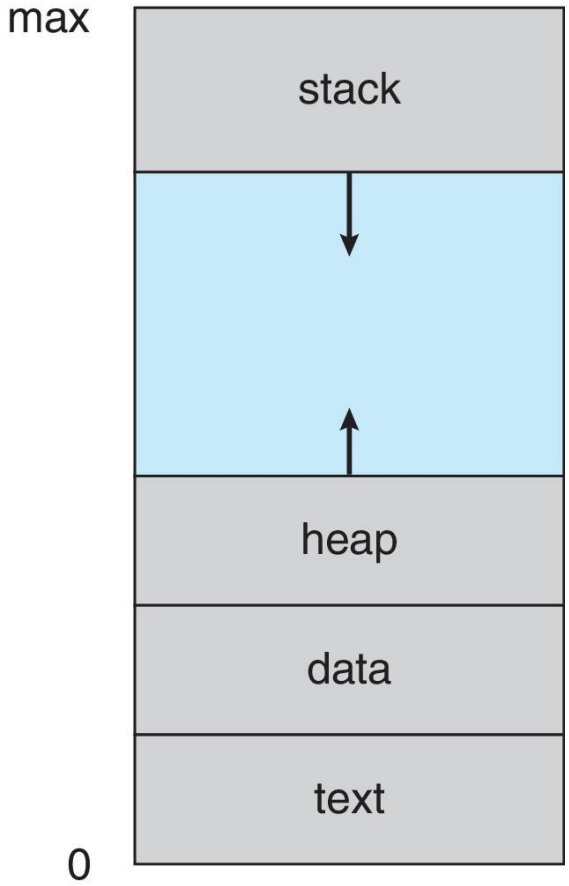
Concepto de proceso (Cont.)

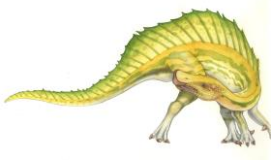
- El programa es una entidad pasiva almacenada en el disco (archivo ejecutable); el proceso está activo
  - El programa se convierte en proceso cuando se carga un archivo ejecutable en la memoria
- La ejecución del programa comenzó a través de clics del mouse de la GUI, entrada de línea de comando de su nombre, etc.
- Un programa puede ser tener varios procesos asociados
  - Considere múltiples usuarios ejecutando el mismo programa





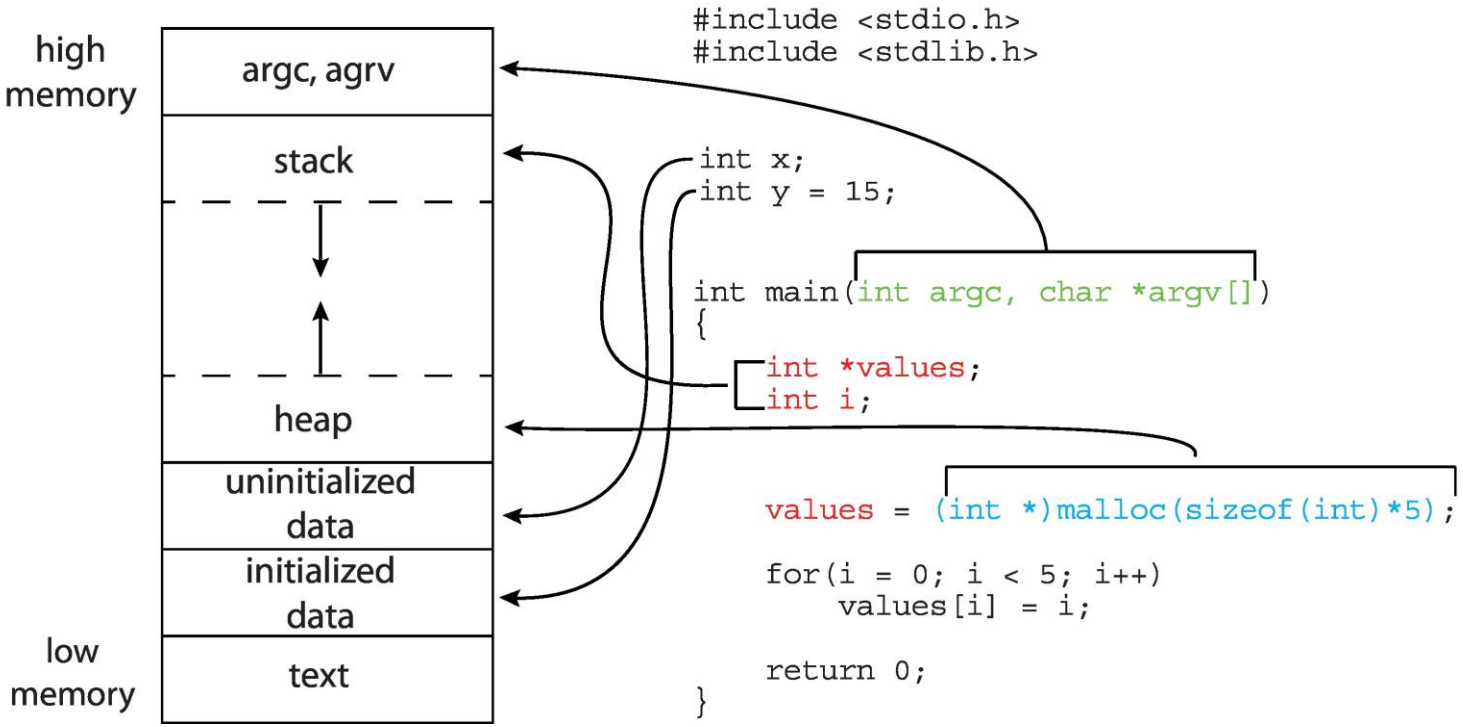
Proceso en memoria







Diseño de memoria de un programa en C







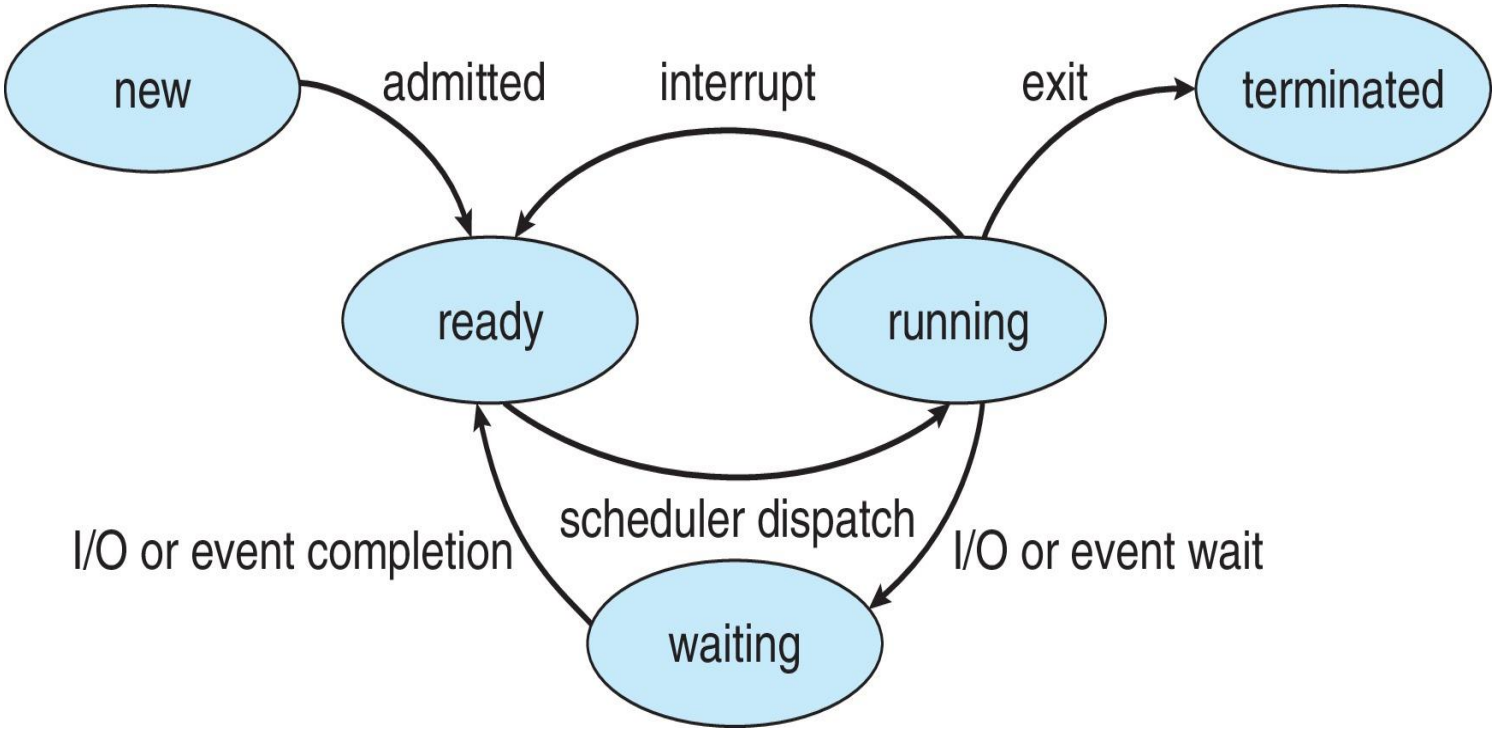
Estado del proceso

- A medida que se ejecuta un proceso, este cambia de estado
  - **Nuevo:** el proceso se está creando
  - **En ejecución:** las instrucciones se están ejecutando
  - **En espera:** el proceso está esperando que ocurra algún evento
  - **Listo:** el proceso está esperando ser asignado a un procesador
  - **Terminado:** el proceso ha finalizado la ejecución





Diagrama del estado del proceso



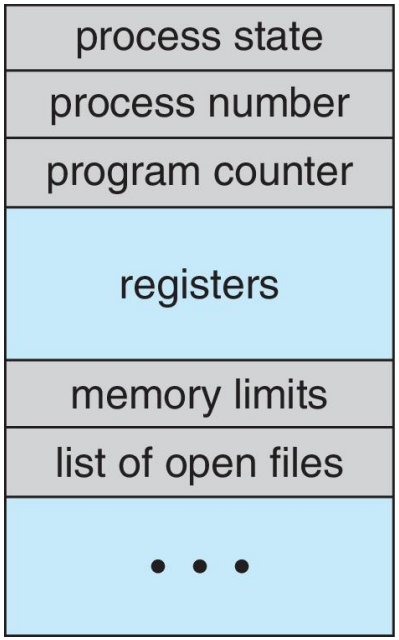


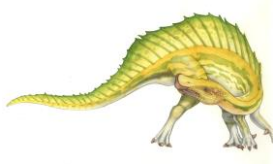


Bloque de control de proceso (PCB)

Información asociada con cada proceso (también llamado bloque de control de tareas)

- Estado del proceso: en ejecución, en espera, etc.
- Contador de programa: ubicación de la instrucción para la próxima ejecución
- Registros de CPU: contenido de todos los registros centrados en el proceso
- Información de programación de CPU: prioridades, programación de punteros de cola
- Información de gestión de memoria: memoria asignada al proceso
- Información contable: CPU utilizada, tiempo transcurrido desde el inicio, límites de tiempo
- Información de estado de E / S: dispositivos de E / S asignados al proceso, lista de archivos abiertos







Hilos

- Hasta ahora, el proceso tiene un solo hilo de ejecución
- Considere tener múltiples contadores de programa por proceso
  - Se pueden ejecutar varias ubicaciones a la vez
    - Múltiples hilos de control -> hilos
- Luego debe tener almacenamiento para detalles de subprocesos, contadores de programas múltiples en PCB





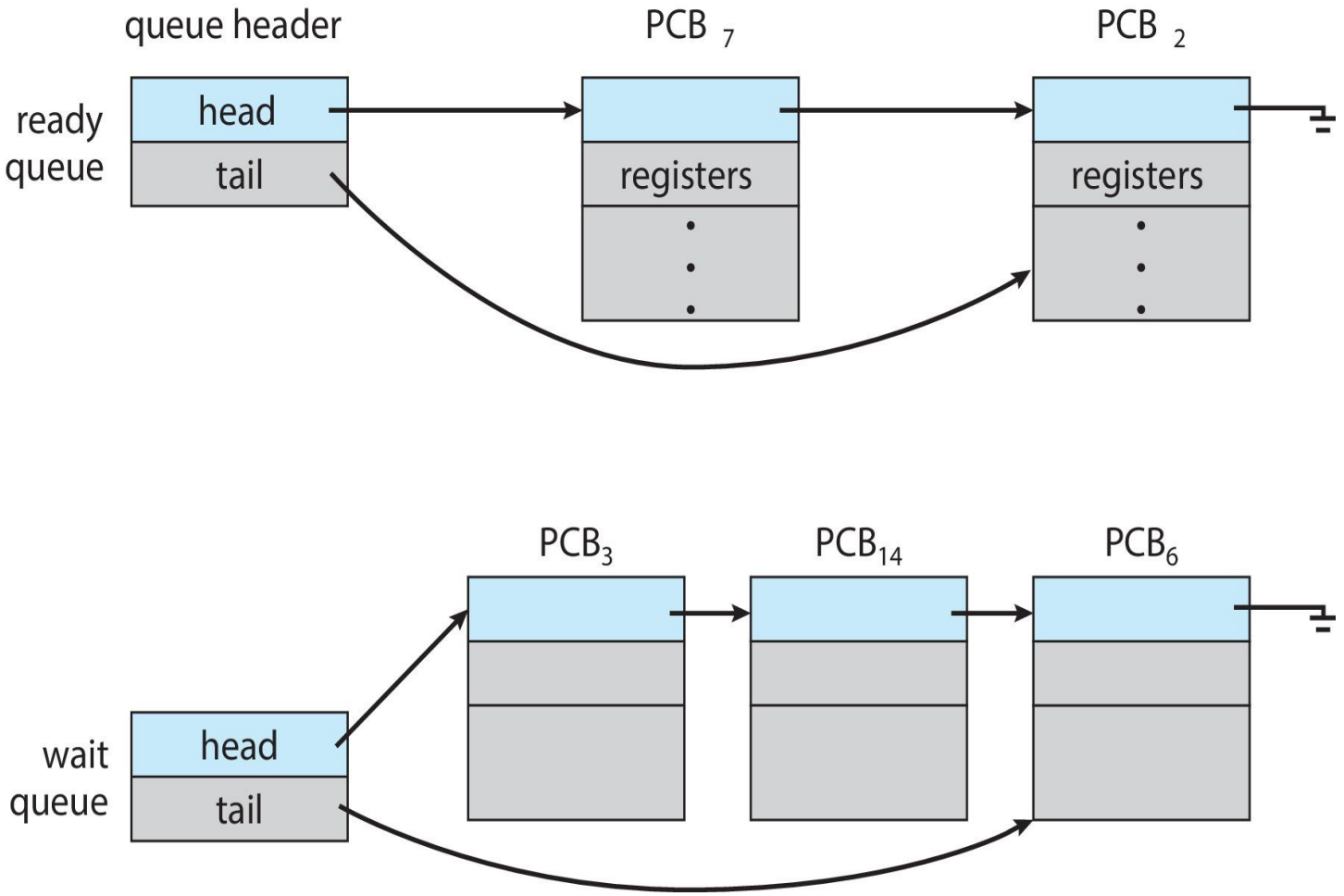
Programación de Procesos

- El planificador de procesos selecciona entre los procesos disponibles para la próxima ejecución en el núcleo de la CPU
- Objetivo: maximizar el uso de la CPU, cambiar rápidamente los procesos al núcleo de la CPU

- Mantiene la programación de colas de procesos
  - Cola de listos: conjunto de todos los procesos que residen en la memoria principal, listos y en espera de ejecución
  - Colas de espera: conjunto de procesos que esperan un evento (es decir, E / S)
  - Los procesos migran entre las diversas colas.

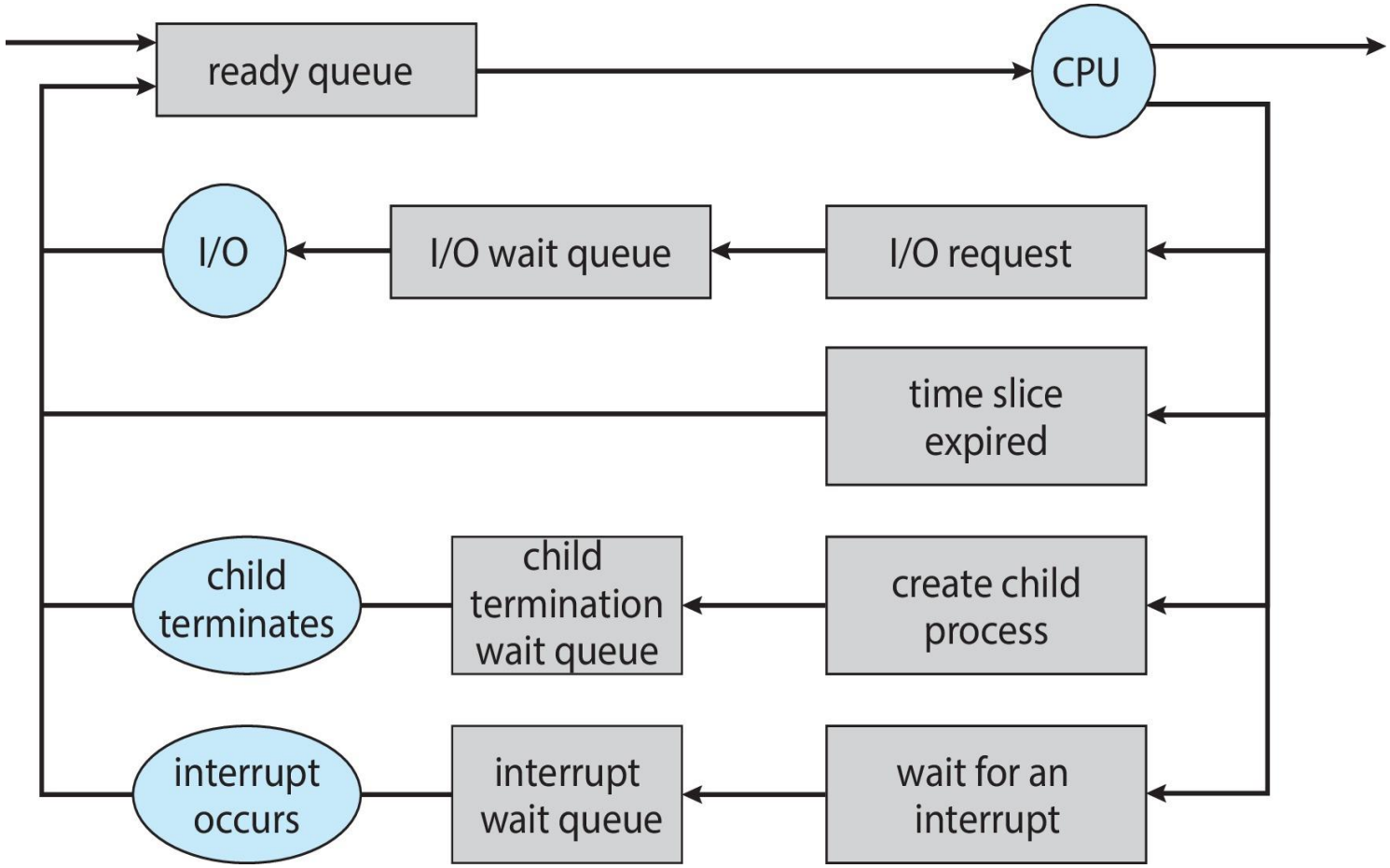


Listo y colas de espera





Representación de la programación del proceso

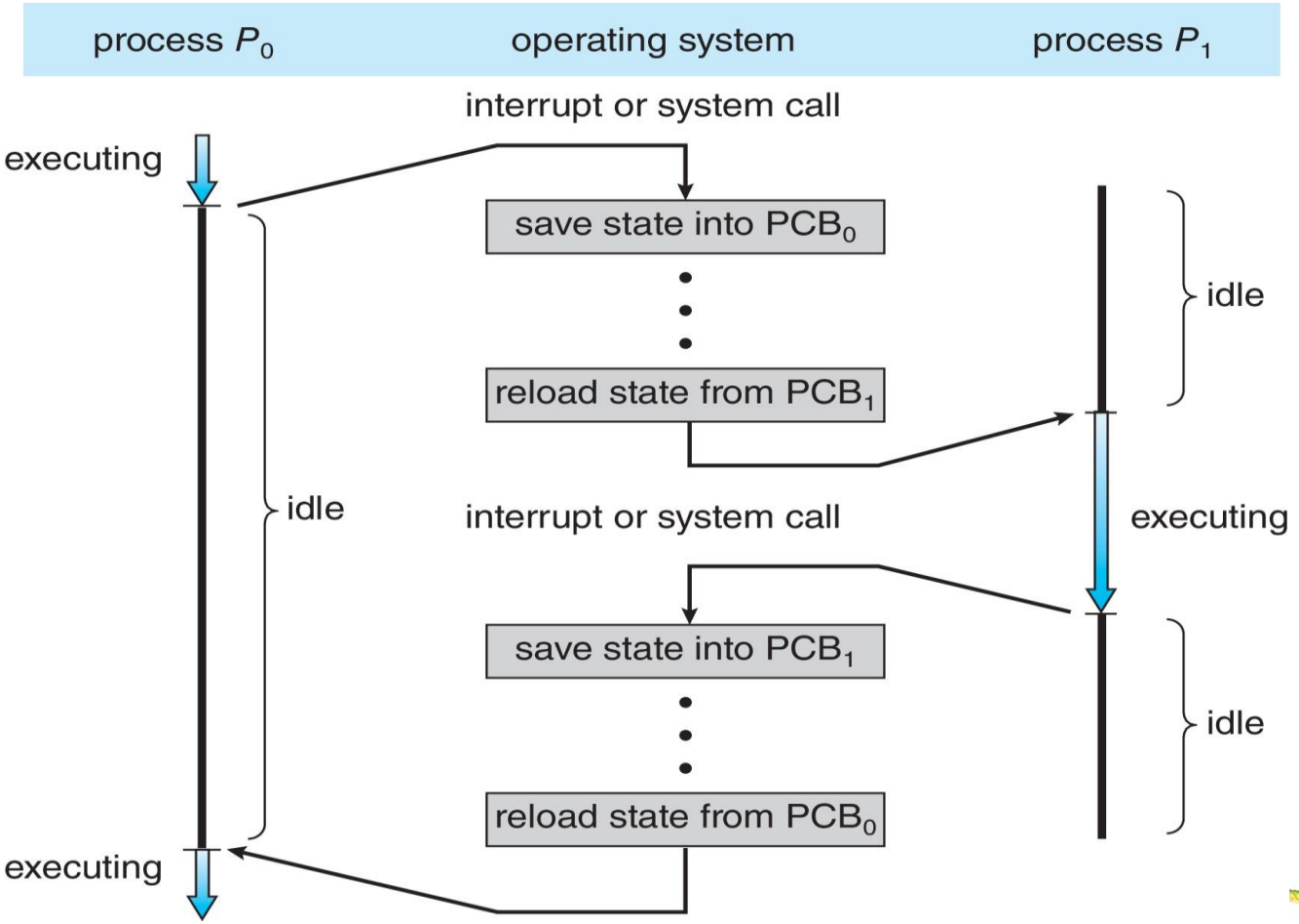






Cambio de CPU de proceso a proceso

Un cambio de contexto ocurre cuando la CPU cambia de un proceso a otro.







Cambio de contexto

- Cuando la CPU cambia a otro proceso, el sistema debe guardar el estado del proceso anterior y cargar el estado guardado para el nuevo proceso a través de un cambio de contexto
- Contexto de un proceso representado en el PCB
- El tiempo de cambio de contexto es de sobrecarga; el sistema no realiza ningún trabajo útil al cambiar

- Cuanto más complejo es el sistema operativo y la PCB, más largo es el cambio de contexto
  - Tiempo dependiente del soporte de hardware
    - Parte del hardware proporciona múltiples conjuntos de registros por CPU
    - múltiples contextos cargados a la vez





Multitarea en sistemas móviles

- Algunos sistemas móviles (por ejemplo, versiones anteriores de iOS) permiten que solo se ejecute un proceso, los otros procesos están suspendidos

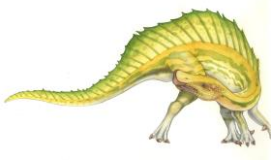
- Debido al estado de la pantalla, los límites de la interfaz de usuario que iOS proporciona
  - Proceso único en primer plano controlado a través de la interfaz de usuario
  - Múltiples procesos en segundo plano: en la memoria, en ejecución, pero no en la pantalla y con límites
  - Los límites incluyen tareas simples y cortas, recibir notificaciones de eventos, tareas específicas de larga duración como reproducción de audio





Multitarea en sistemas móviles

- Android se ejecuta en primer plano y en segundo plano, con menos límites
  - El proceso en segundo plano utiliza un servicio para realizar tareas
  - El servicio puede seguir ejecutándose incluso si se suspende el proceso en segundo plano
  - El servicio no tiene interfaz de usuario, uso de memoria pequeña





Operaciones en procesos

- El sistema debe proporcionar mecanismos para:
  - Proceso de creación
  - Terminación del proceso.





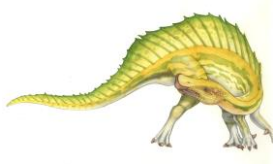
Creación del Proceso

- El proceso padre crea procesos hijos, que a su vez crean otros procesos, formando un árbol de procesos.

- En general, el proceso se identifica y se gestiona mediante un identificador de proceso (pid)

- Opciones para compartir recursos
  - Padres e hijos comparten todos los recursos
  - Los hijos comparten un subconjunto de los recursos de los padres
  - Padre e hijo no comparten recursos

- Opciones de ejecución
  - Padres e hijos se ejecutan simultáneamente
  - Los padres esperan hasta que los hijos terminen

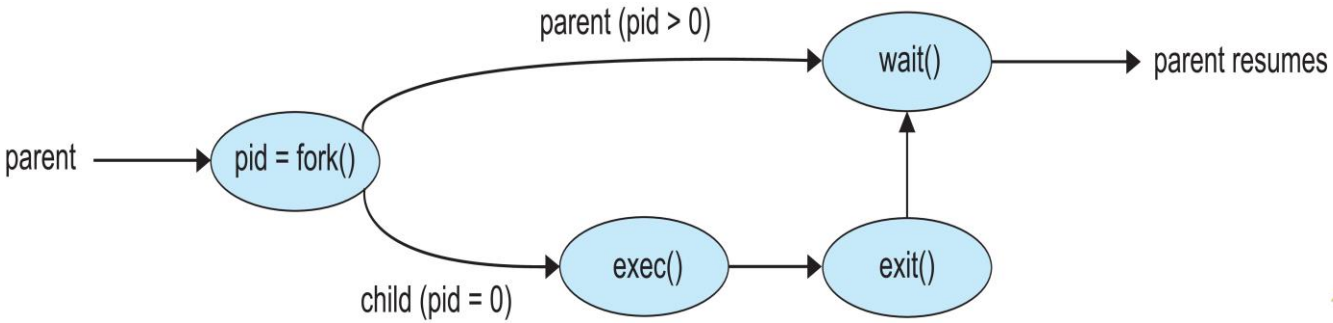


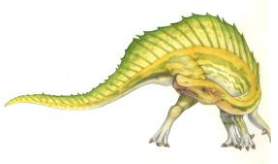


Creación de procesos (Cont.)

- Espacio de dirección
  - Niño duplicado de padre
  - El niño tiene un programa cargado

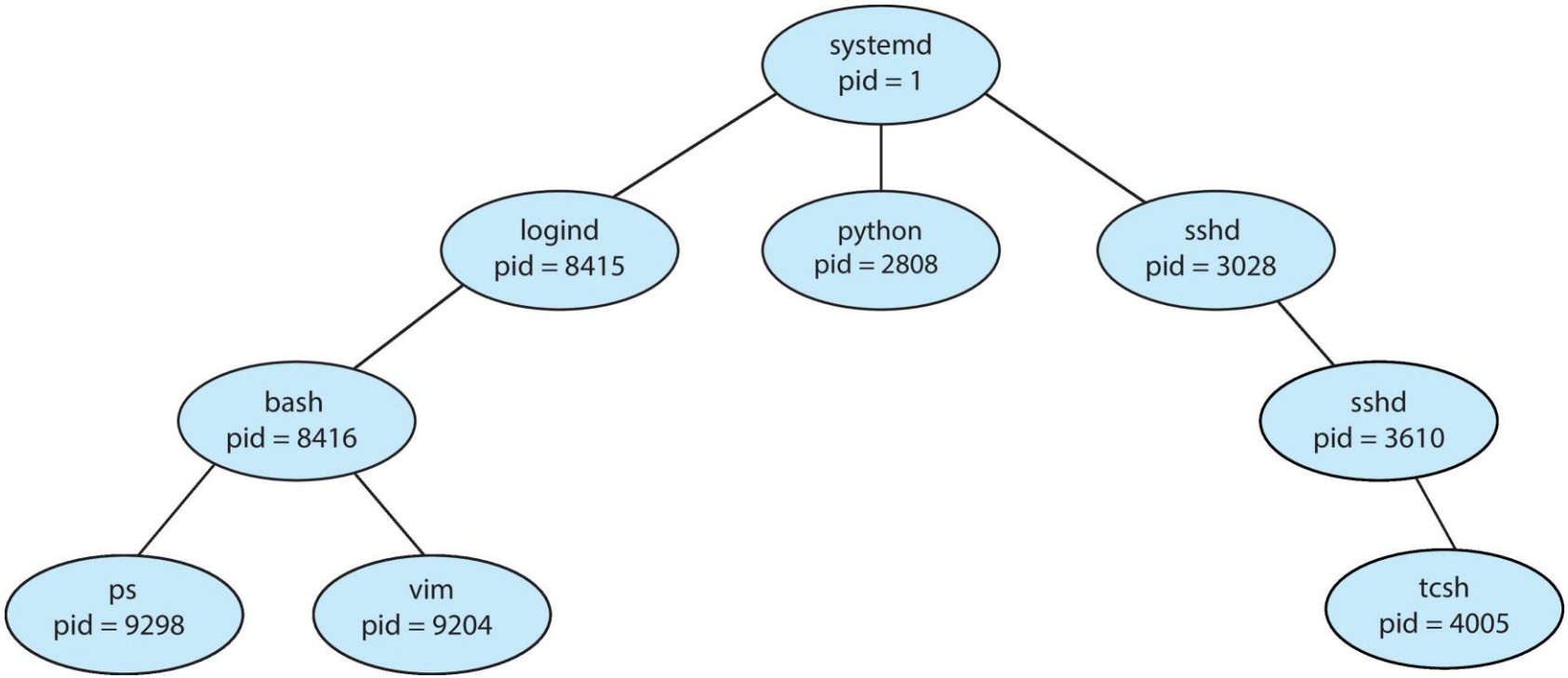
- Ejemplos de UNIX
  - **fork ()** La llamada al sistema crea un nuevo proceso
  - **exec ()** La llamada al sistema utilizada después de un fork () para reemplazar el espacio de memoria del proceso con un nuevo programa
  - El proceso padre llama a **wait ()** esperando que el hijo termine







Un árbol de procesos en Linux







C Program Forking Separate Process

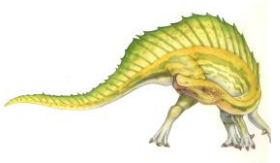
```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
       fprintf(stderr, "Fork Failed");
       return 1;
    }
    else if (pid == 0) { /* child process */
       execlp("/bin/ls","ls",NULL);
    }
    else { /* parent process */
       /* parent will wait for the child to complete */
       wait(NULL);
       printf("Child Complete");
    }

    return 0;
}
```





Crear un proceso a través de la API de Windows

```
#include <stdio.h>
#include <windows.h>

int main(VOID)
{
STARTUPINFO si;
PROCESS_INFORMATION pi;

    /* allocate memory */
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));

    /* create child process */
    if (!CreateProcess(NULL, /* use command line */
     "C:\\WINDOWS\\system32\\mspaint.exe", /* command */
     NULL, /* don't inherit process handle */
     NULL, /* don't inherit thread handle */
     FALSE, /* disable handle inheritance */
     0, /* no creation flags */
     NULL, /* use parent's environment block */
     NULL, /* use parent's existing directory */
     &si,
     &pi))
    {
      fprintf(stderr, "Create Process Failed");
      return -1;
    }
    /* parent will wait for the child to complete */
    WaitForSingleObject(pi.hProcess, INFINITE);
    printf("Child Complete");

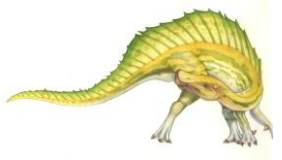
    /* close handles */
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
}
```





Terminación de Procesos

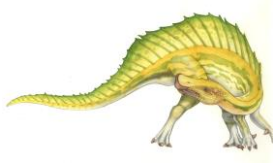
- El proceso ejecuta la última instrucción y luego le pide al sistema operativo que la elimine mediante la llamada al sistema exit ().
  - Devuelve datos de estado del hijo al padre (a través de wait ())
  - Los recursos del proceso se desasignan por sistema operativo
- El padre puede terminar la ejecución de los procesos hijos utilizando la llamada al sistema **abort ()**. Algunas razones para hacerlo:
  - El hijo ha excedido los recursos asignados
  - La tarea asignada al hijo ya no es necesaria
  - El padre está saliendo y los sistemas operativos no permiten que un hijo continúe si su padre termina





Terminación de Procesos







Process Termination

- Some operating systems do not allow child to exists if its parent has terminated. If a process terminates, then all its children must also be terminated.
  - **cascading termination.** All children, grandchildren, etc., are terminated.
  - The termination is initiated by the operating system.
- The parent process may wait for termination of a child process by using the **`wait()`** system call. The call returns status information and the pid of the terminated process

```
pid = wait(&status);
```

- If no parent waiting (did not invoke **`wait()`**) process is a **zombie**
- If parent terminated without invoking **`wait()`**, process is an **orphan**





Android Process Importance Hierarchy

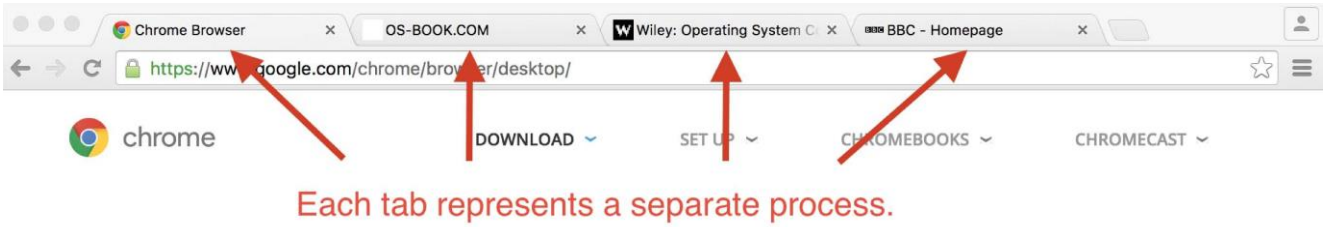
- Mobile operating systems often have to terminate processes to reclaim system resources such as memory. From **most** to **least** important:
  - Foreground process
  - Visible process
  - Service process
  - Background process
  - Empty process
- Android will begin terminating processes that are least important.

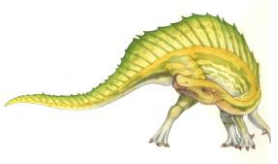




Multiprocess Architecture – Chrome Browser

- Many web browsers ran as single process (some still do)
  - If one web site causes trouble, entire browser can hang or crash
- Google Chrome Browser is multiprocess with 3 different types of processes:
  - **Browser** process manages user interface, disk and network I/O
  - **Renderer** process renders web pages, deals with HTML, Javascript. A new renderer created for each website opened
    - Runs in **sandbox** restricting disk and network I/O, minimizing effect of security exploits
  - **Plug-in** process for each type of plug-in







Interprocess Communication

- Processes within a system may be ***independent*** or ***cooperating***
- Cooperating process can affect or be affected by other processes, including sharing data
- Reasons for cooperating processes:
  - Information sharing
  - Computation speedup
  - Modularity
  - Convenience
- Cooperating processes need **interprocess communication** (**IPC**)
- Two models of IPC
  - **Shared memory**
  - **Message passing**

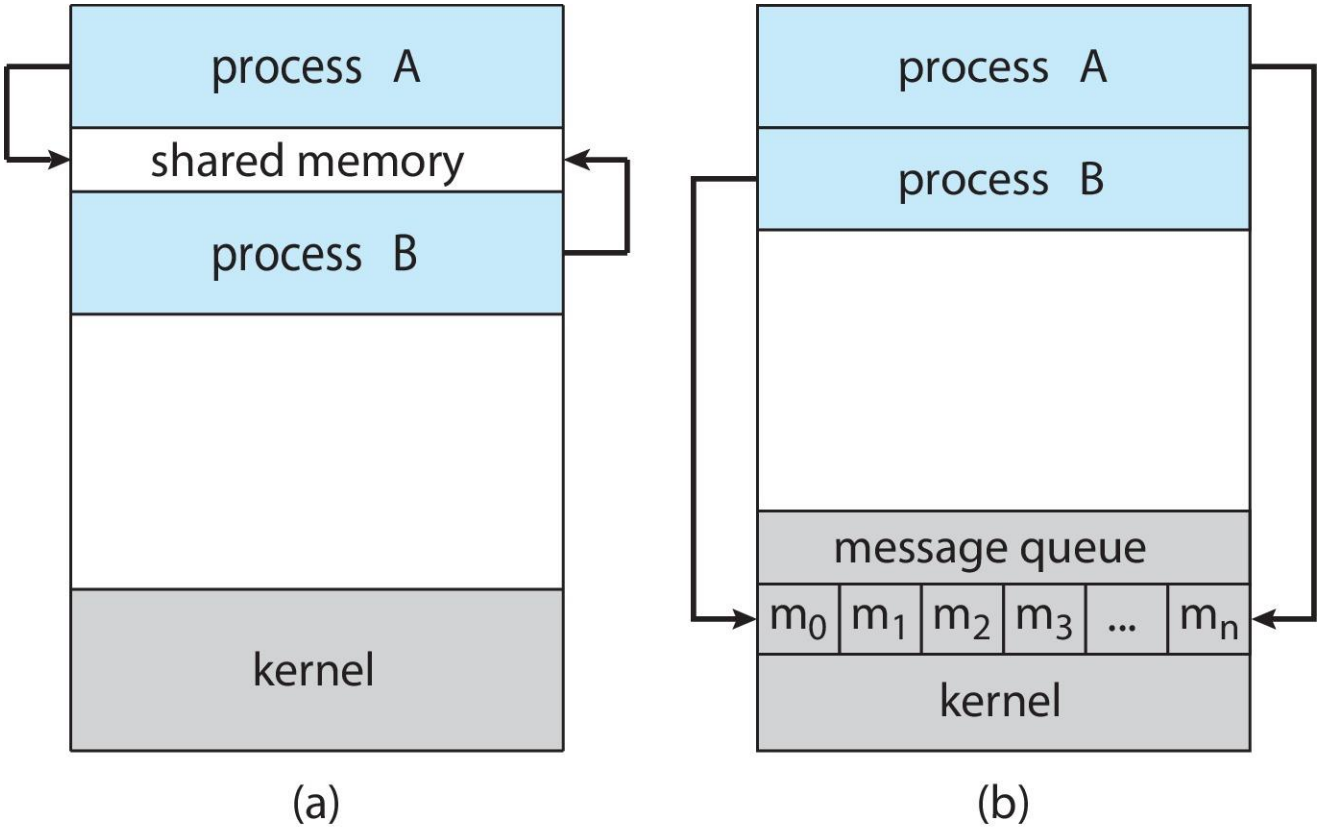




Communications Models

(a) Shared memory.

(b) Message passing.







Producer-Consumer Problem

- Paradigm for cooperating processes:
  - *producer* process produces information that is consumed by a *consumer* process
- Two variations:
  - **unbounded-buffer** places no practical limit on the size of the buffer:
    - Producer never waits
    - Consumer waits if there is no buffer to consume
  - **bounded-buffer** assumes that there is a fixed buffer size
    - Producer must wait if all buffers are full
    - Consumer waits if there is no buffer to consume





IPC – Shared Memory

- An area of memory shared among the processes that wish to communicate
- The communication is under the control of the users processes not the operating system.
- Major issues is to provide mechanism that will allow the user processes to synchronize their actions when they access shared memory.
- Synchronization is discussed in great details in Chapters 6 & 7.





Bounded-Buffer – Shared-Memory Solution

- Shared data

```
#define BUFFER_SIZE 10
typedef struct {
  . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

- Solution is correct, but can only use **`BUFFER_SIZE-1`** elements





Producer Process – Shared Memory

```
item next_produced;

while (true) {
   /* produce an item in next produced */
   while (((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing */
   buffer[in] = next_produced;
   in = (in + 1) % BUFFER_SIZE;
}
```





Consumer Process – Shared Memory

```
item next_consumed;

while (true) {
        while (in == out)
                ; /* do nothing */
        next_consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;

        /* consume the item in next consumed */
}
```





What about Filling all the Buffers?

- Suppose that we wanted to provide a solution to the consumer-producer problem that fills **all** the buffers.
- We can do so by having an integer **`counter`** that keeps track of the number of full buffers.
- Initially, **`counter`** is set to 0.
- The integer **`counter`** is incremented by the producer after it produces a new buffer.
- The integer **`counter`** is and is decremented by the consumer after it consumes a buffer.





Producer

```
while (true) {
        /* produce an item in next produced */

        while (counter == BUFFER_SIZE)
                ; /* do nothing */
        buffer[in] = next_produced;
        in = (in + 1) % BUFFER_SIZE;
        counter++;
}
```





Consumer

```
while (true) {
        while (counter == 0)
                ; /* do nothing */
        next_consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
         counter--;
        /* consume the item in next consumed */
}
```





Race Condition

- **`counter++`** could be implemented as

```
register1 = counter
register1 = register1 + 1
counter = register1
```

- **`counter--`** could be implemented as

```
register2 = counter
register2 = register2 - 1
counter = register2
```

- Consider this execution interleaving with "count = 5" initially:

S0: producer execute `register1 = counter` {register1 = 5}
S1: producer execute `register1 = register1 + 1` {register1 = 6}
S2: consumer execute `register2 = counter` {register2 = 5}
S3: consumer execute `register2 = register2 - 1` {register2 = 4}
S4: producer execute `counter = register1` {counter = 6 }
S5: consumer execute `counter = register2` {counter = 4}





Race Condition (Cont.)

- Question – why was there no race condition in the first solution (where at most N – 1) buffers can be filled?
- More in Chapter 6.





IPC – Message Passing

- Processes communicate with each other without resorting to shared variables
- IPC facility provides two operations:
  - **`send`**(*message*)
  - **`receive`**(*message*)
- The *message* size is either fixed or variable





Message Passing (Cont.)

- If processes *P* and *Q* wish to communicate, they need to:
  - Establish a ***communication link*** between them
  - Exchange messages via send/receive
- Implementation issues:
  - How are links established?
  - Can a link be associated with more than two processes?
  - How many links can there be between every pair of communicating processes?
  - What is the capacity of a link?
  - Is the size of a message that the link can accommodate fixed or variable?
  - Is a link unidirectional or bi-directional?





Implementation of Communication Link

- Physical:
  - Shared memory
  - Hardware bus
  - Network
- Logical:
  - Direct or indirect
  - Synchronous or asynchronous
  - Automatic or explicit buffering





Direct Communication

- Processes must name each other explicitly:
  - **`send`** (*P, message*) – send a message to process P
  - **`receive`**(*Q, message*) – receive a message from process Q
- Properties of communication link
  - Links are established automatically
  - A link is associated with exactly one pair of communicating processes
  - Between each pair there exists exactly one link
  - The link may be unidirectional, but is usually bi-directional





Indirect Communication

- Messages are directed and received from mailboxes (also referred to as ports)
  - Each mailbox has a unique id
  - Processes can communicate only if they share a mailbox
- Properties of communication link
  - Link established only if processes share a common mailbox
  - A link may be associated with many processes
  - Each pair of processes may share several communication links
  - Link may be unidirectional or bi-directional





Indirect Communication (Cont.)

- Operations
  - Create a new mailbox (port)
  - Send and receive messages through mailbox
  - Delete a mailbox
- Primitives are defined as:
  - **`send`**(*A, message*) – send a message to mailbox A
  - **`receive`**(*A, message*) – receive a message from mailbox A





Indirect Communication (Cont.)

- Mailbox sharing
  - P_1, P_2, and P_3 share mailbox A
  - P_1, sends; P_2 and P_3 receive
  - Who gets the message?
- Solutions
  - Allow a link to be associated with at most two processes
  - Allow only one process at a time to execute a receive operation
  - Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.

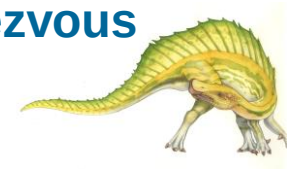




Synchronization

Message passing may be either blocking or non-blocking

- **Blocking** is considered **synchronous**
  - **Blocking send** -- the sender is blocked until the message is received
  - **Blocking receive** -- the receiver is blocked until a message is available
- **Non-blocking** is considered **asynchronous**
  - **Non-blocking send** -- the sender sends the message and continue
  - **Non-blocking receive** -- the receiver receives:
    - A valid message, or
    - Null message
- Different combinations possible
  - If both send and receive are blocking, we have a **rendezvous**





Producer-Consumer: Message Passing

- Producer

```
message next_produced;
while (true) {
  /* produce an item in next_produced */

  send(next_produced);
}
```

- Consumer

```
message next_consumed;
while (true) {
  receive(next_consumed)

  /* consume the item in next_consumed */
}
```





Buffering

- Queue of messages attached to the link.
- Implemented in one of three ways
  1. Zero capacity – no messages are queued on a link.
     Sender must wait for receiver (rendezvous)
  2. Bounded capacity – finite length of *n* messages
     Sender must wait if link full
  3. Unbounded capacity – infinite length
     Sender never waits





Examples of IPC Systems - POSIX

- POSIX Shared Memory
  - Process first creates shared memory segment
    `shm_fd = shm_open(name, O CREAT | O RDWR, 0666);`
  - Also used to open an existing segment
  - Set the size of the object

    `ftruncate(shm_fd, 4096);`
  - Use **`mmap()`** to memory-map a file pointer to the shared memory object
  - Reading and writing to shared memory is done by using the pointer returned by **`mmap()`**.





IPC POSIX Producer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
/* the size (in bytes) of shared memory object */
const int SIZE = 4096;
/* name of the shared memory object */
const char *name = "OS";
/* strings written to shared memory */
const char *message_0 = "Hello";
const char *message_1 = "World!";

/* shared memory file descriptor */
int shm_fd;
/* pointer to shared memory obect */
void *ptr;

   /* create the shared memory object */
   shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);

   /* configure the size of the shared memory object */
   ftruncate(shm_fd, SIZE);

   /* memory map the shared memory object */
   ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);

   /* write to the shared memory object */
   sprintf(ptr,"%s",message_0);
   ptr += strlen(message_0);
   sprintf(ptr,"%s",message_1);
   ptr += strlen(message_1);

   return 0;
}
```





IPC POSIX Consumer

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
/* the size (in bytes) of shared memory object */
const int SIZE = 4096;
/* name of the shared memory object */
const char *name = "OS";
/* shared memory file descriptor */
int shm_fd;
/* pointer to shared memory obect */
void *ptr;

    /* open the shared memory object */
    shm_fd = shm_open(name, O_RDONLY, 0666);

    /* memory map the shared memory object */
    ptr = mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);

    /* read from the shared memory object */
    printf("%s",(char *)ptr);

    /* remove the shared memory object */
    shm_unlink(name);

    return 0;
}
```





Examples of IPC Systems - Mach

- Mach communication is message based
  - Even system calls are messages
  - Each task gets two ports at creation - Kernel and Notify
  - Messages are sent and received using the **`mach_msg()`** function
  - Ports needed for communication, created via

    `mach_port_allocate()`
  - Send and receive are flexible; for example four options if mailbox full:
    - Wait indefinitely
    - Wait at most n milliseconds
    - Return immediately
    - Temporarily cache a message





Mach Messages

```
#include<mach/mach.h>

struct message {
        mach_msg_header_t header;
        int data;
};

mach port t client;
mach port t server;
```





Mach Message Passing - Client

```
/* Client Code */

struct message message;

// construct the header
message.header.msgh_size = sizeof(message);
message.header.msgh_remote_port = server;
message.header.msgh_local_port = client;

// send the message
mach_msg(&message.header, // message header
   MACH_SEND_MSG, // sending a message
   sizeof(message), // size of message sent
   0, // maximum size of received message - unnecessary
   MACH_PORT_NULL, // name of receive port - unnecessary
   MACH_MSG_TIMEOUT_NONE, // no time outs
   MACH_PORT_NULL // no notify port
);
```





Mach Message Passing - Server

```
        /* Server Code */

struct message message;

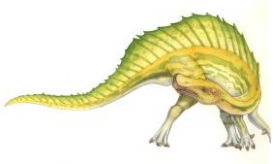
// receive the message
mach_msg(&message.header, // message header
   MACH_RCV_MSG, // sending a message
   0, // size of message sent
   sizeof(message), // maximum size of received message
   server, // name of receive port
   MACH_MSG_TIMEOUT_NONE, // no time outs
   MACH_PORT_NULL // no notify port
);
```





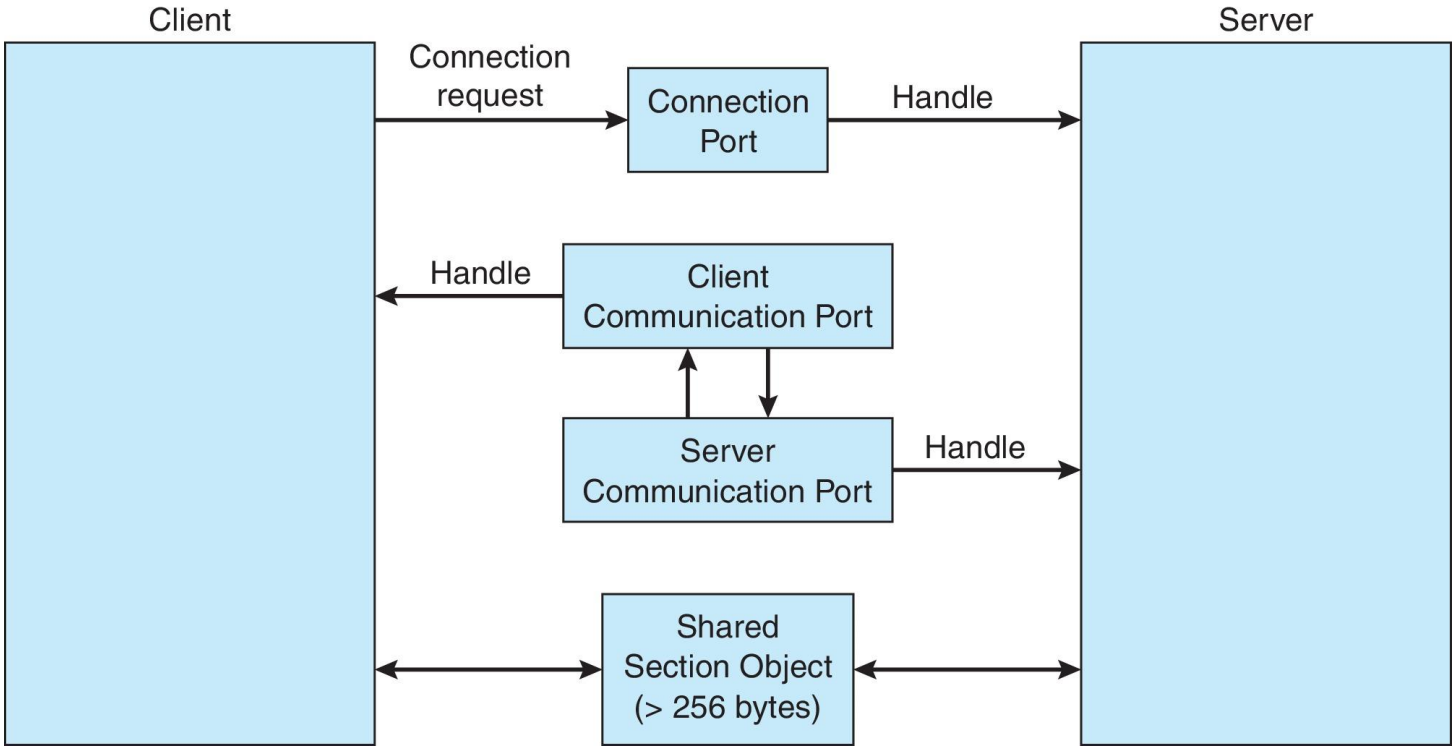
Examples of IPC Systems – Windows

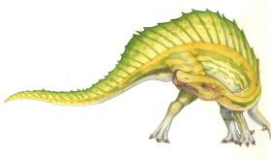
- Message-passing centric via **advanced local procedure call (LPC)** facility
  - Only works between processes on the same system
  - Uses ports (like mailboxes) to establish and maintain communication channels
  - Communication works as follows:
    - The client opens a handle to the subsystem's **connection port** object.
    - The client sends a connection request.
    - The server creates two private **communication ports** and returns the handle to one of them to the client.
    - The client and server use the corresponding port handle to send messages or callbacks and to listen for replies.





Local Procedure Calls in Windows







Pipes

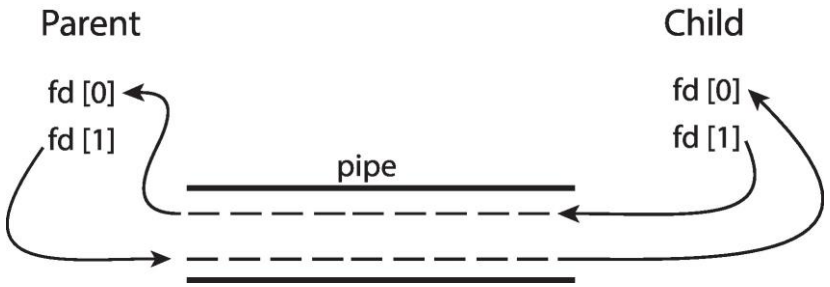
- Acts as a conduit allowing two processes to communicate
- Issues:
  - Is communication unidirectional or bidirectional?
  - In the case of two-way communication, is it half or full-duplex?
  - Must there exist a relationship (i.e., ***parent-child***) between the communicating processes?
  - Can the pipes be used over a network?
- **Ordinary pipes** – cannot be accessed from outside the process that created it. Typically, a parent process creates a pipe and uses it to communicate with a child process that it created.
- **Named pipes** – can be accessed without a parent-child relationship.



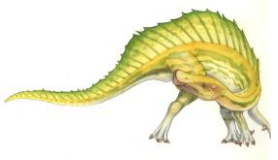


Ordinary Pipes

- Ordinary Pipes allow communication in standard producer-consumer style
- Producer writes to one end (the **write-end** of the pipe)
- Consumer reads from the other end (the **read-end** of the pipe)
- Ordinary pipes are therefore unidirectional
- Require parent-child relationship between communicating processes



- Windows calls these **anonymous pipes**





Named Pipes

- Named Pipes are more powerful than ordinary pipes
- Communication is bidirectional
- No parent-child relationship is necessary between the communicating processes
- Several processes can use the named pipe for communication
- Provided on both UNIX and Windows systems





Communications in Client-Server Systems

- Sockets
- Remote Procedure Calls





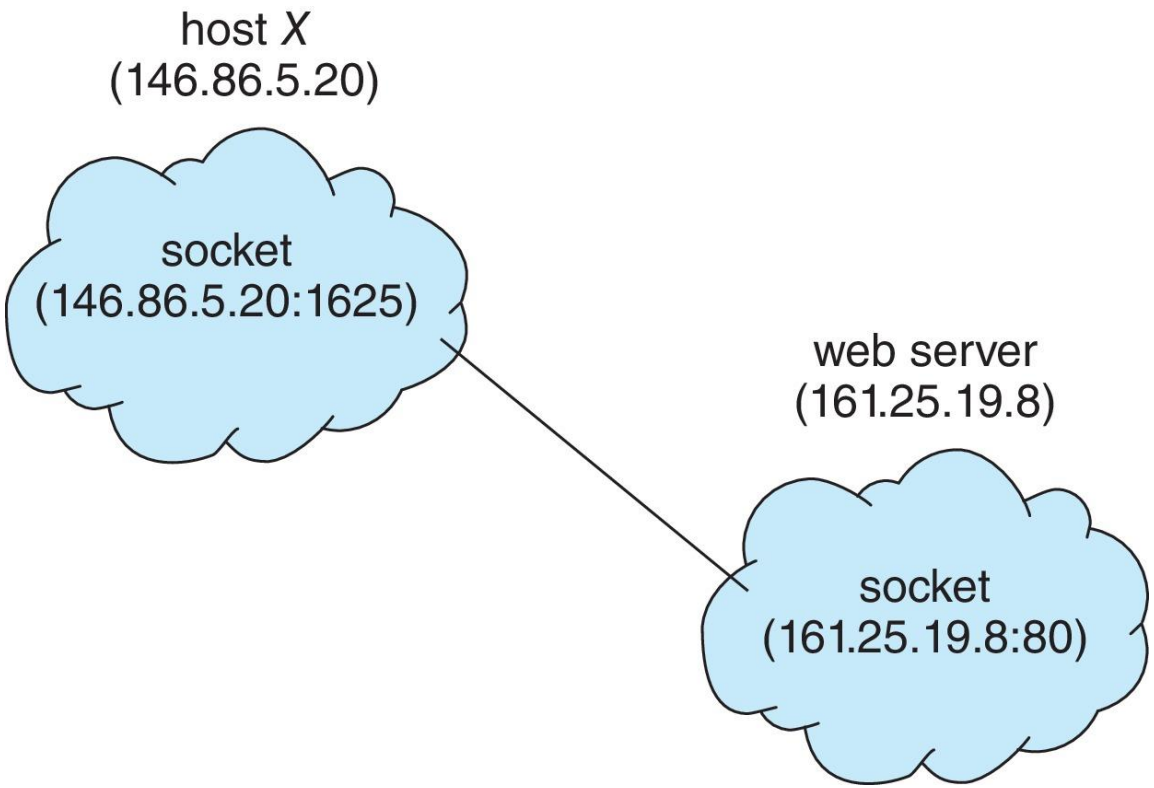
Sockets

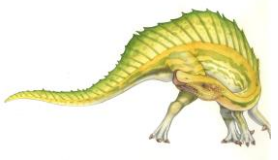
- A **socket** is defined as an endpoint for communication
- Concatenation of IP address and **port** – a number included at start of message packet to differentiate network services on a host
- The socket **161.25.19.8:1625** refers to port **1625** on host **161.25.19.8**
- Communication consists between a pair of sockets
- All ports below 1024 are ***well known***, used for standard services
- Special IP address 127.0.0.1 (**loopback**) to refer to system on which process is running





Socket Communication







Sockets in Java

- Three types of sockets
  - **Connection-oriented** (**TCP**)
  - **Connectionless** (**UDP**)
  - **`MulticastSocket`** class– data can be sent to multiple recipients
- Consider this “Date” server in Java:

```
import java.net.*;
import java.io.*;

public class DateServer
{
   public static void main(String[] args) {
      try {
         ServerSocket sock = new ServerSocket(6013);

         /* now listen for connections */
         while (true) {
            Socket client = sock.accept();

            PrintWriter pout = new
             PrintWriter(client.getOutputStream(), true);

            /* write the Date to the socket */
            pout.println(new java.util.Date().toString());

            /* close the socket and resume */
            /* listening for connections */
            client.close();
         }
      }
      catch (IOException ioe) {
         System.err.println(ioe);
      }
   }
}
```





Sockets in Java

The equivalent Date client

```
import java.net.*;
import java.io.*;

public class DateClient
{
   public static void main(String[] args) {
      try {
         /* make connection to server socket */
         Socket sock = new Socket("127.0.0.1",6013);

         InputStream in = sock.getInputStream();
         BufferedReader bin = new
            BufferedReader(new InputStreamReader(in));

         /* read the date from the socket */
         String line;
         while ( (line = bin.readLine()) != null)
            System.out.println(line);

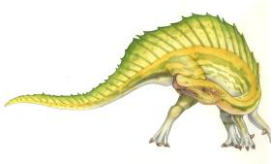
         /* close the socket connection*/
         sock.close();
      }
      catch (IOException ioe) {
         System.err.println(ioe);
      }
   }
}
```





Remote Procedure Calls

- Remote procedure call (RPC) abstracts procedure calls between processes on networked systems
  - Again uses ports for service differentiation
- **Stubs** – client-side proxy for the actual procedure on the server
- The client-side stub locates the server and **marshalls** the parameters
- The server-side stub receives this message, unpacks the marshalled parameters, and performs the procedure on the server
- On Windows, stub code compile from specification written in **Microsoft Interface Definition Language** (**MIDL**)





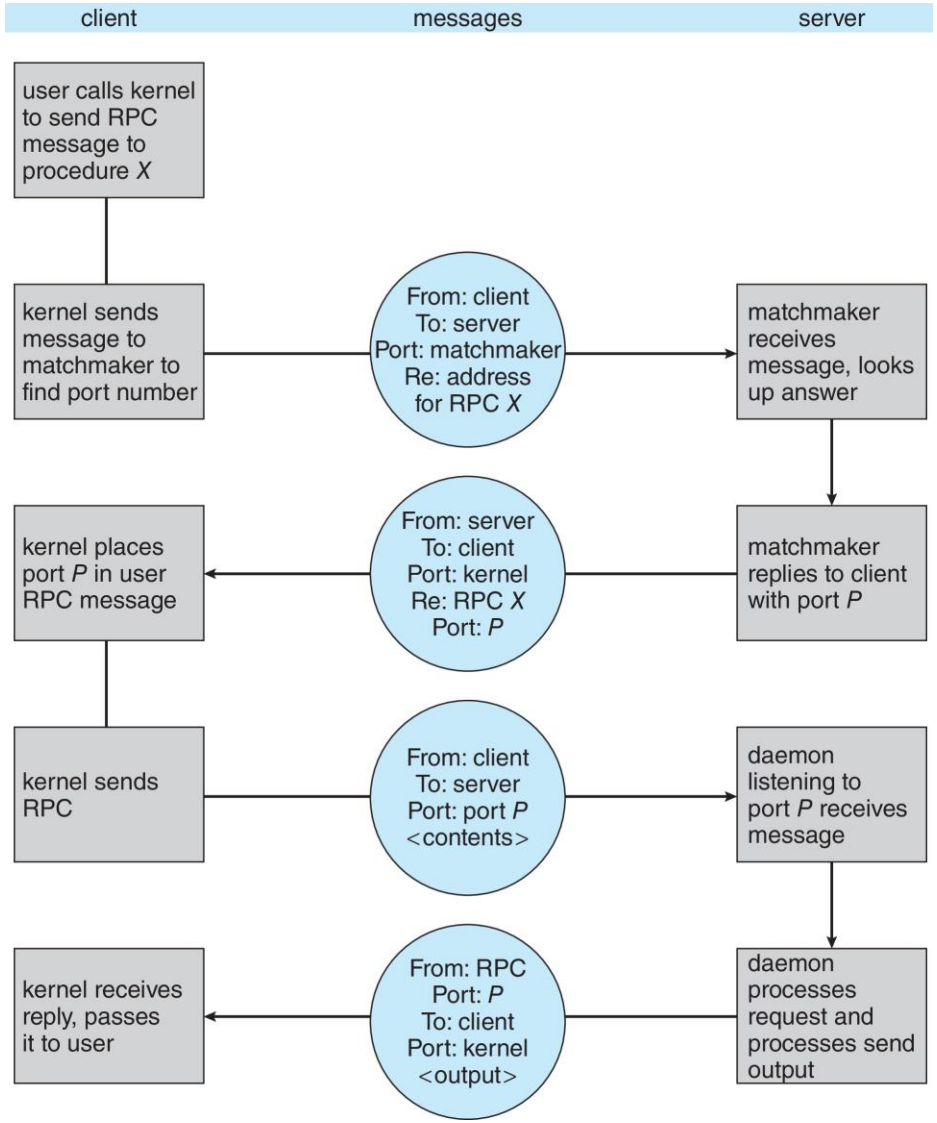
Remote Procedure Calls (Cont.)

- Data representation handled via **External Data Representation** (**XDL**) format to account for different architectures
  - **Big-endian** and **little-endian**
- Remote communication has more failure scenarios than local
  - Messages can be delivered ***exactly once*** rather than ***at most once***
- OS typically provides a rendezvous (or **matchmaker**) service to connect client and server



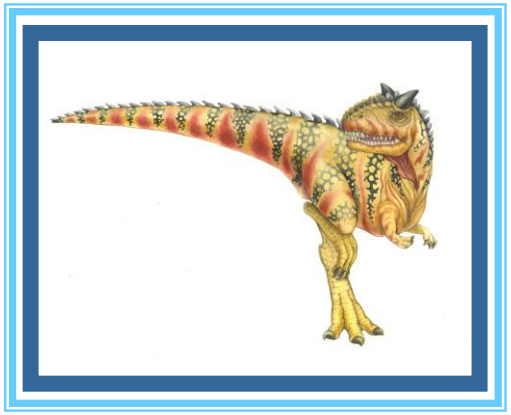


Execution of RPC





End of Chapter 3



Silberschatz, Galvin and Gagne ©2018



Producer-Consumer Problem

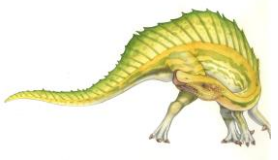
- Paradigm for cooperating processes:
  - *producer* process produces information that is consumed by a *consumer* process
- Two variations:
  - **unbounded-buffer** places no practical limit on the size of the buffer:
    - Producer never waits
    - Consumer waits if there is no buffer to consume
  - **bounded-buffer** assumes that there is a fixed buffer size
    - Producer must wait if all buffers are full
    - Consumer waits if there is no buffer to consume





Cooperating Processes

- ***Independent*** process cannot affect or be affected by the execution of another process
- ***Cooperating*** process can affect or be affected by the execution of another process
- Advantages of process cooperation
  - Information sharing
  - Computation speed-up
  - Modularity
  - Convenience





Synchronization

- Message passing may be either blocking or non-blocking
  - **Blocking** is considered **synchronous**
    - **Blocking send** -- the sender is blocked until the message is received
    - **Blocking receive** -- the receiver is blocked until a message is available
  - **Non-blocking** is considered **asynchronous**
    - **Non-blocking send** -- the sender sends the message and continue
    - **Non-blocking receive** -- the receiver receives:
    - A valid message, or
    - Null message
  - Different combinations possible
    - If both send and receive are blocking, we have a **rendezvous**

